AVR 기반 초음파 자율주행 RC카

ATmega128A와 초음파 센서를 활용한 지능형 주행 시스템

고지현 곽혜민

목차

01 프로젝트 개요

02 요소 기술

03 회로도

04 STD(상태전이도)

05 코드 설명

06 동작 영상

07 소감

프로젝트 개요

프로젝트 목적

AVR의 ATmega128A 마이크로컨트롤러에 대한 이해

BUTTON, FND, PWM, UART, BT, ULTRASONIC 등을 프로젝트에 활용

프로젝트 목표

ATmega128A와 각종 소자, 센서, 통신 방식을 활용하여 주변 장애물을 감지해
스스로 주행 방향을 결정하는 **자율주행 RC카를 구현**하고
주어진 코스를 큰 충돌 없이 완주하는 것.

주요 기능

수동 주행

- Bluetooth UART 통신
- 스마트폰 앱 제어
- 전후좌우 방향 제어
- 실시간 명령 전송

장애물 감지

- HC-SR04 센서 3개
- 전방·좌·우 측정
- 초음파 활용

자율주행

- 거리 비교 알고리즘
- 최적 경로 자동 선택
- 장애물 회피 주행
- 실시간 방향 결정

속도 제어

- PWM 신호 생성
- 모터 드라이버 제어

상태 표시

- 4-Digit FND 출력
- 주행 모드 표시
- 거리 정보 표시
- 주행 기록 저장

주요 부품

번호	부품명	부품
0	ATmega128A Demo Board	MCU 제어 보드
1	HC-SR04 × 3	초음파 센서
2	Bluetooth Module	무선 통신 모듈
3	DC Motor × 4	차량 구동 모터
4	Motor Driver	모터 구동 모듈
5	4-Digit FND * 2	주행 정보 표시장치
6	Tact Switch	입력 스위치

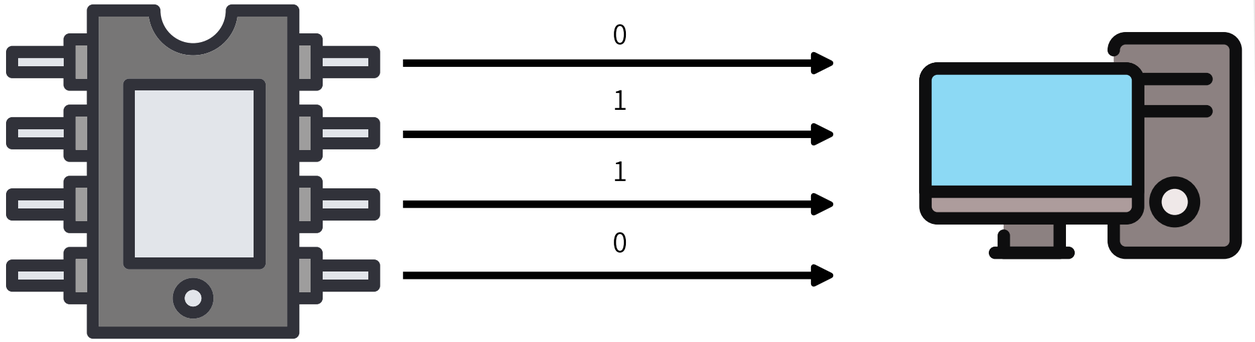
UART(Universal Asynchronous Receiver/Transmitter)

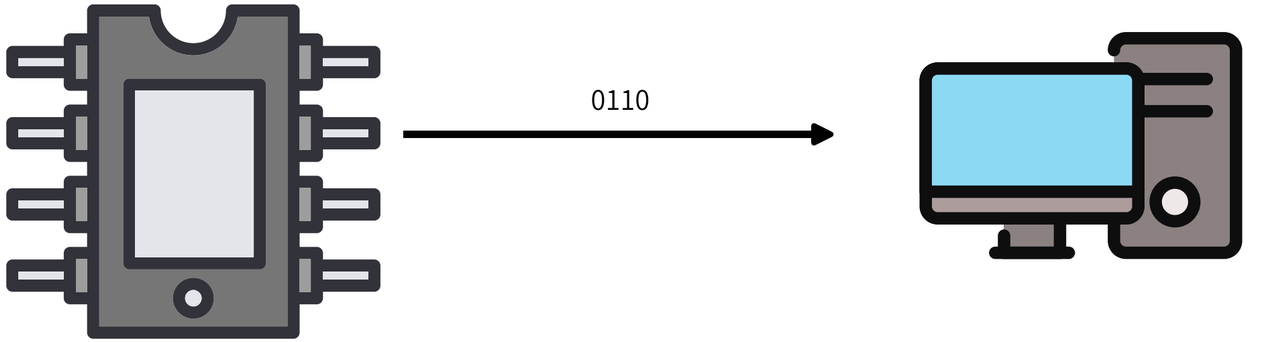
병렬(Parallel) 통신

- 여러 개의 데이터 비트를 동시에 전송하는 방식
- n bit 데이터를 전송할 때 n 개의 선이 필요
- 사용 예시: CPU <->메모리 버스, LCD 데이터 버스 등

직렬(Serial) 통신

- 데이터를 1 비트씩 순서대로 전송하는 방식
- 데이터 전송 시 하나의 데이터 선만 필요.
- 사용 예시: UART, SPI, I2C 등 MCU에서 주로 사용 (MCU는 GPIO 핀이 많지 않기 때문)





UART(Universal Asynchronous Receiver/Transmitter)

구분	병렬 통신	직렬 통신
전송 방식	n 개의 핀으로 n bit 데이터를 한 번에	n bit 데이터를 1 bit씩 순차적으로
데이터 라인 수	n 개	1 개
전송 속도	상대적으로 빠름	상대적으로 느림
회로 복잡도	복잡함	간단함
비용	비용 ↑	비용 ↓
장거리 전송	장거리 전송 불리	장거리 전송 유리
노이즈 영향	노이즈 영향이 크다.	노이즈 영향이 적다.
사용 예시	메모리 버스	UART, SPI, I2C

UART 주요 특징

송수신 속도

- 보율(Baud Rate)

연결 방법

- 거리 비교 알고리즘
- 최적 경로 자동 선택
- 장애물 회피 주행
- 실시간 방향 결정

데이터 형태

- Idle B
- Start Bit
- Stop Bit
- Parity Bit(선택)

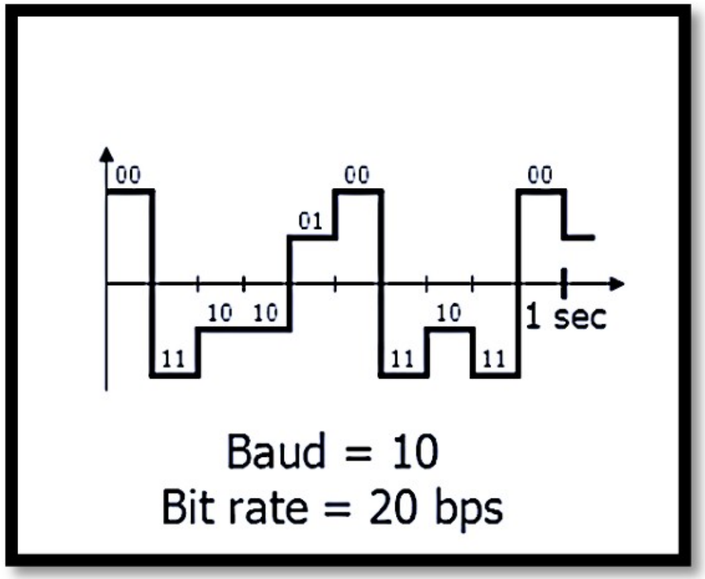
관련 레지스터와 핵심 Bit

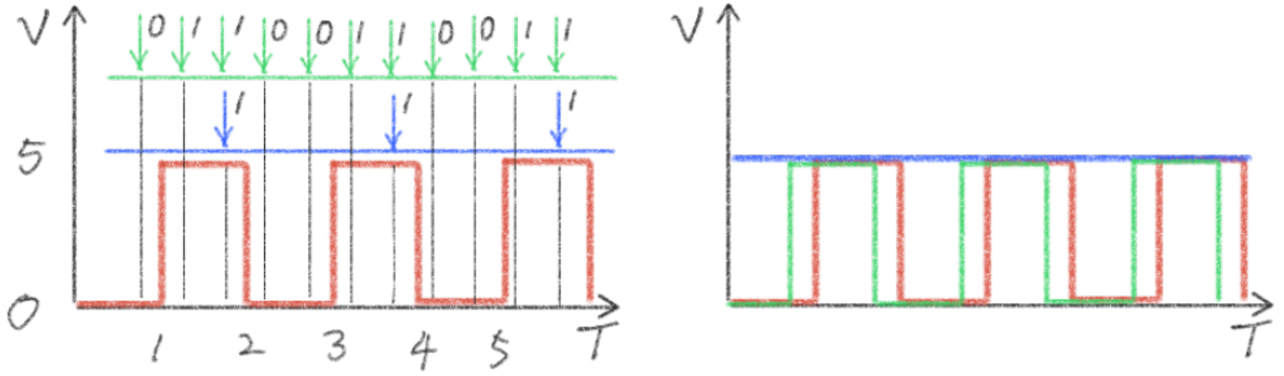
- UBRRn
- UCSRnx
- UDRn

UART(Universal Asynchronous Receiver/Transmitter)

송수신 속도

- 송신 속도와 수신 속도를 맞춰야 데이터를 올바르게 주고받을 수 있다.
- 이 때 데이터를 주고 받는 속도를 나타내는 것이 바로 보율(Baud Rate: 1초 동안 전달되는 데이터 비트의 개수).
- 송신기와 수신기의 Baud Rate가 같아야 정상 통신이 가능하다.





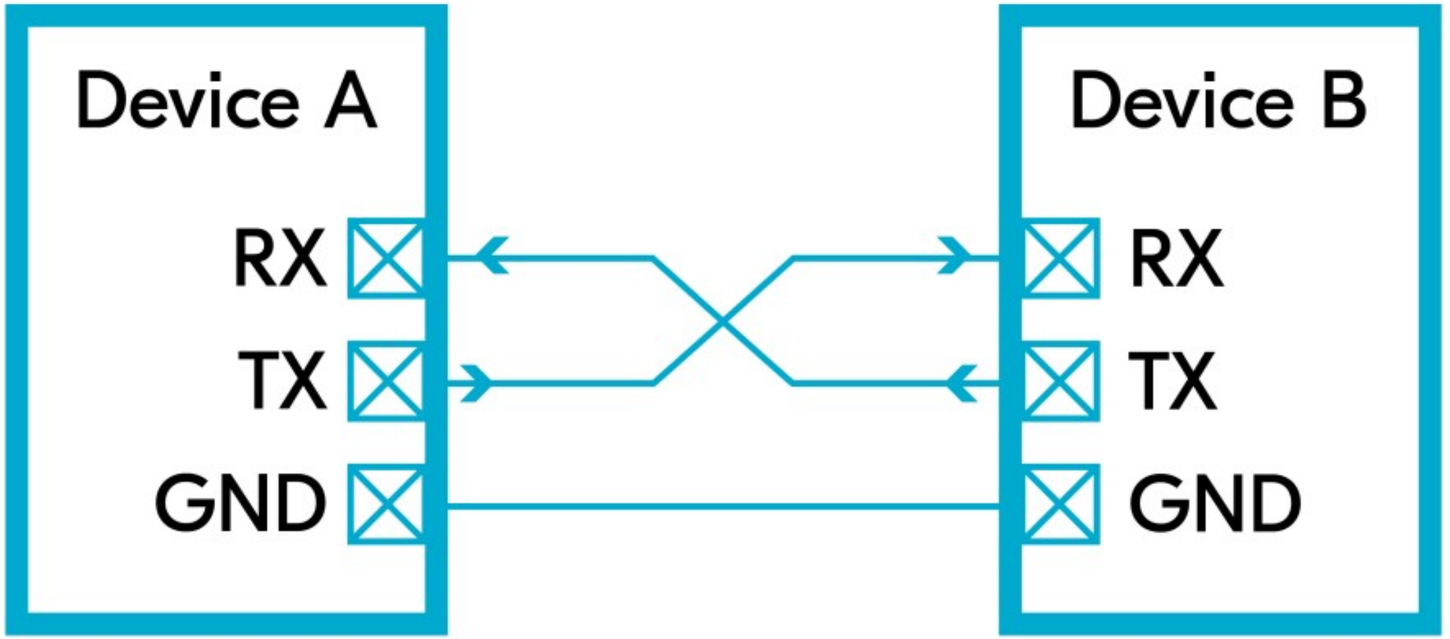
(출처2: https://kwonkyo.tistory.com/593#gsc.tab=0)

(출처1: https://rahsoft.com/2024/11/30/understanding-bit-rate-baud-rate-and-their-importance-in-communication-systems/) (출처2: https://kwonkyo.tistory.com/593#gsc.tab=0)

UART(Universal Asynchronous Receiver/Transmitter)

연결 방법

- 송신기와 수신기의 TX, RX를 교차로 연결
- ex) 송신기의 TX를 수신기의 RX로, 송신기의 RX를 수신기의 TX로

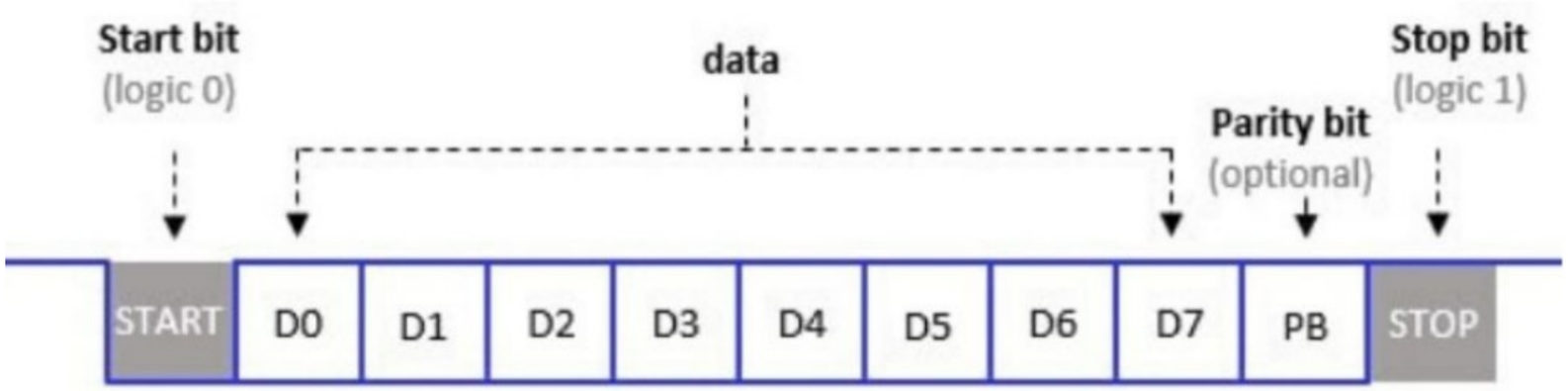


(출처: https://academy.nordicsemi.com/courses/nrf-connect-sdk-fundamentals/lessons/lesson-4-serial-communication-uart/topic/uart-protocol/)

UART(Universal Asynchronous Receiver/Transmitter)

데이터 형태

- Idle Bit: 데이터를 송/수신하지 않을 때 데이터 핀의 초기 값('1')
- Start Bit: 통신이 시작됨을 알린다. ('1' → '0')
- Data Bit: 실제 전송할 데이터 (ex. 8 bits)
- Parity Bit: Parity Error를 검출하기 위한 Bit (Even: 1의 개수가 짝수. Odd: 1의 개수가 홀수)
- Stop Bit: 데이터 전송이 끝났음을 알린다. ('X' → '1')



(출처: https://velog.io/@sang_yun_911/CS-UART%ED%86%B5%EC%8B%A0)

UART(Universal Asynchronous Receiver/Transmitter)

관련 레지스터와 핵심 Bit (w. ATmega128A)

1. UCSRnx(USART Control Status Register n x, n = 0, 1 / x = A, B, C)
   - UCSRnA
     ★ bit 7) RXCn, 6) TXCn, 5) UDREn, 1) U2Xn
   - UCSRnB
     ★ 7) RXCIEn, 6) TXCIEn, 5) UDRIEn, 4) RXENn, 3) TXENn, UCSZn2
   - UCSRnC
     ★ 7) UMSELn, 3) USBS, 2, 1) UCSZn0, UCSZn1

2. UBRRnH/L(USART Baud Rate Register)(n = 0, 1) : 보율을 16 bit로 표현 (UBRRnH → 상위 4 bit, UBRRnL → 하위 8 bit)

3. UDRn(USART Data Register(n = 0, 1) : 송수신 데이터 저장

Bluetooth

개인 근거리 무선 통신(Personal Area Network, PAN)

- 기본적으로 10 m 이내에서의 통신을 목표로 하고, 최대 100 m 까지 확장 가능

2.4 GHz 대역인 ISM(Industrial, Scientific, Medical) 대역 사용

- 상위 및 하위 영역은 다른 시스템과의 간섭을 막기 위해 사용 X. 2402 MHz ~ 2480 MHz까지 총 79개 채널 사용

Master & Slave 구조

- 하나의 마스터에 최대 7개까지 슬레이브 연결 가능
- 마스터 <--> 슬레이브 통신 O. But, 슬레이브 <--> 슬레이브 통신 X.

사용 예

- 스마트폰, 마우스 , 키보드, 이어폰, 프린터, 카메라, 자동차, 게임기, TV 등

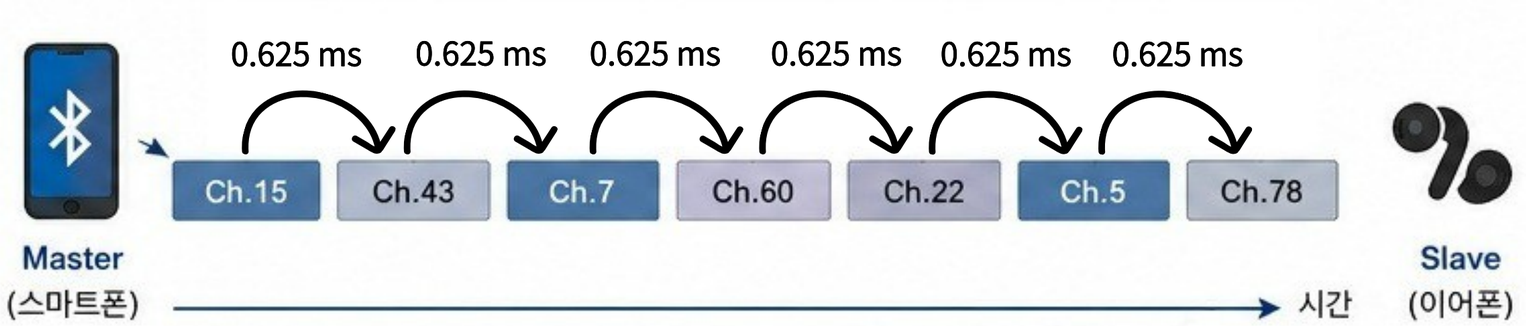
Bluetooth - Frequency Hoping

Bluetooth 주파수 호핑이란?

- 하나의 주파수에 머무르지 않고 여러 주파수(채널)을 매우 빠르게 번갈아 사용하며 데이터를 전송하는 방식

- 1600 hops/sec로 주파수 호핑을 한다고 가정하면 약 0.625 ms 마다 다른 채널로 이동

- 송/수신기가 동일한 호핑 순서를 알고 있어야 한다.

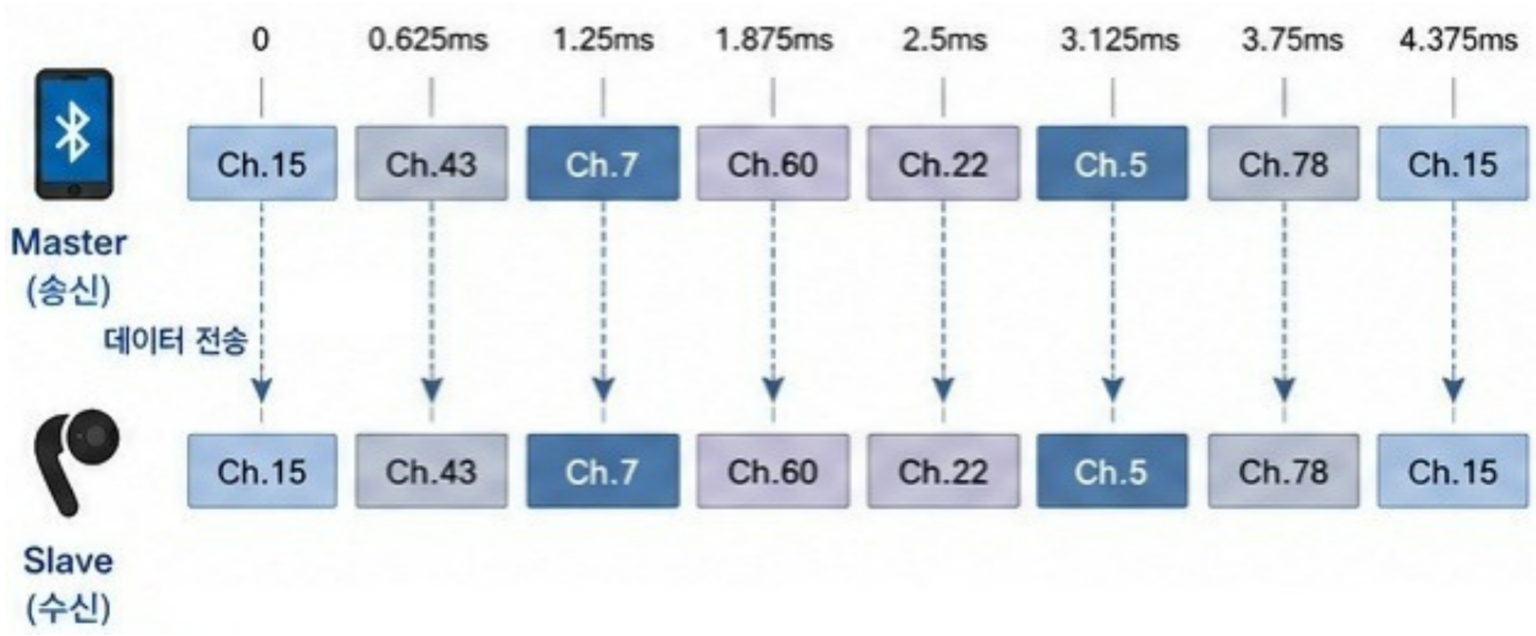


Bluetooth - Frequency Hoping

Bluetooth 주파수 호핑 동작 원리

- 마스터와 슬레이브가 동일한 호핑 시퀀스를 따라 채널을 변경.

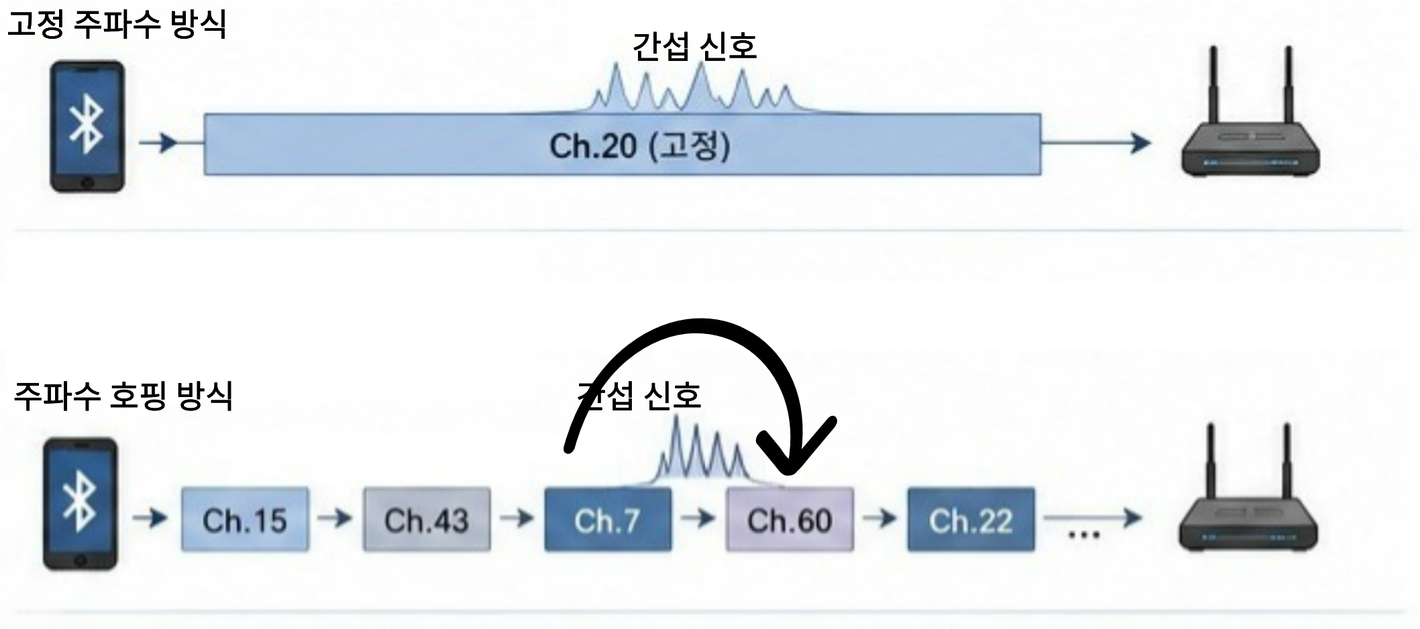
호핑 시퀀스	
순서	channel
1	→ Ch.15
2	→ Ch.43
3	→ Ch.7
4	→ Ch.60
5	→ Ch.22
6	→ Ch.5
7	→ Ch.78
8	→ Ch.15



Bluetooth - Frequency Hoping

Bluetooth 주파수 호핑의 장점

- 특정 채널이 간섭이 있어도 다른 채널로 이동하여 계속 통신가능
- 일부 채널을 감청해도 전체 내용을 알기 어려워 보안성 향상



Bluetooth - Frequency Hoping

AFH: Adaptive Frequency Hoping

- 간섭이 심한 채널은 제외하고, 깨끗한 채널만 선택하여 사용
- 장치가 주변 환경을 측정하여 간섭 채널을 동적으로 판단하고 호핑 테이블을 갱신



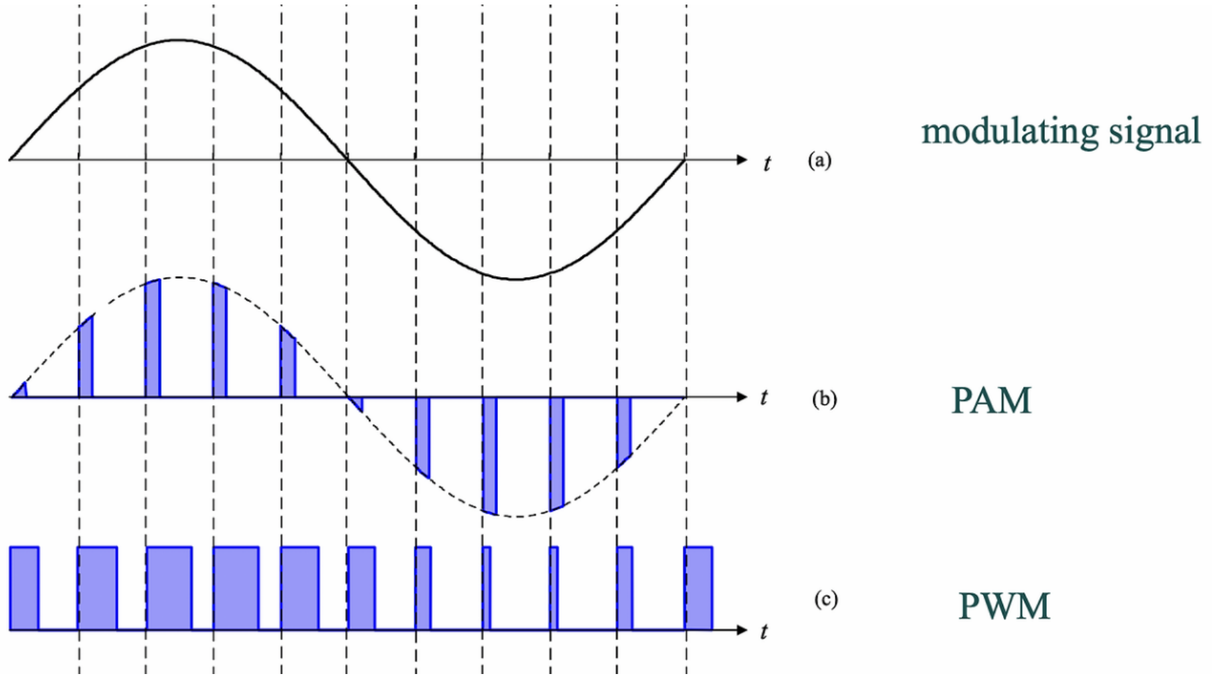
PAM & PWM

PAM(Pulse Amplitude Modulation)

- 펄스의 높이(진폭)를 변화시켜 정보를 전달하는 방식
- 사용 예: 초기 디지털 통신, ADC, 광통신

PWM(Pulse Width Modulation)

- PWM은 펄스의 폭(Duty Cycle)을 변화시켜 정보를 전달
- 사용 예: LED 밝기 제어, 모터 제어, SMPS 전원공급장치, 인버터



(출처: https://sharpcoder.tistory.com/137)

PAM & PWM

구분	PAM	PWM
펄스 변화 요소	진폭(Amplitude)	펄스의 폭(Width, Duty Cycle)
펄스 높이	변함	일정
펄스 폭	일정	변함
펄스 주기	일정	변할 수 있음
노이즈	노이즈에 취약(MSB 값이 바뀌면 치명적)	노이즈에 강함(0과 1)
전력 효율	효율 ↓	효율 ↑ (스위칭 동작)
구현 난이도	상대적으로 쉬움(진폭만 바꾸면 됨)	상대적으로 어려움
모터 제어 적합도	<<	
LED 제어 적합도	<<	
MCU 활용	<<	

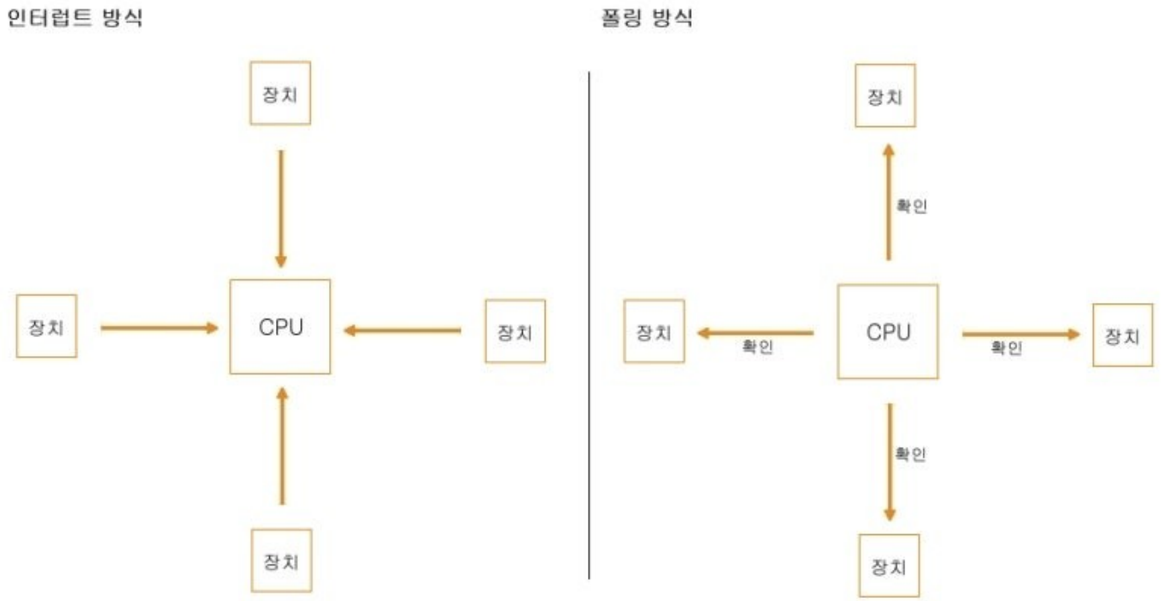
Polling & Interrupt

Polling

- CPU가 주기적으로 장치의 상태를 확인하여 이벤트 발생 여부를 검사하는 방식

Intterrupt

- Interrupt는 이벤트가 발생했을 때 장치가 CPU에게 직접 알려주는 방식

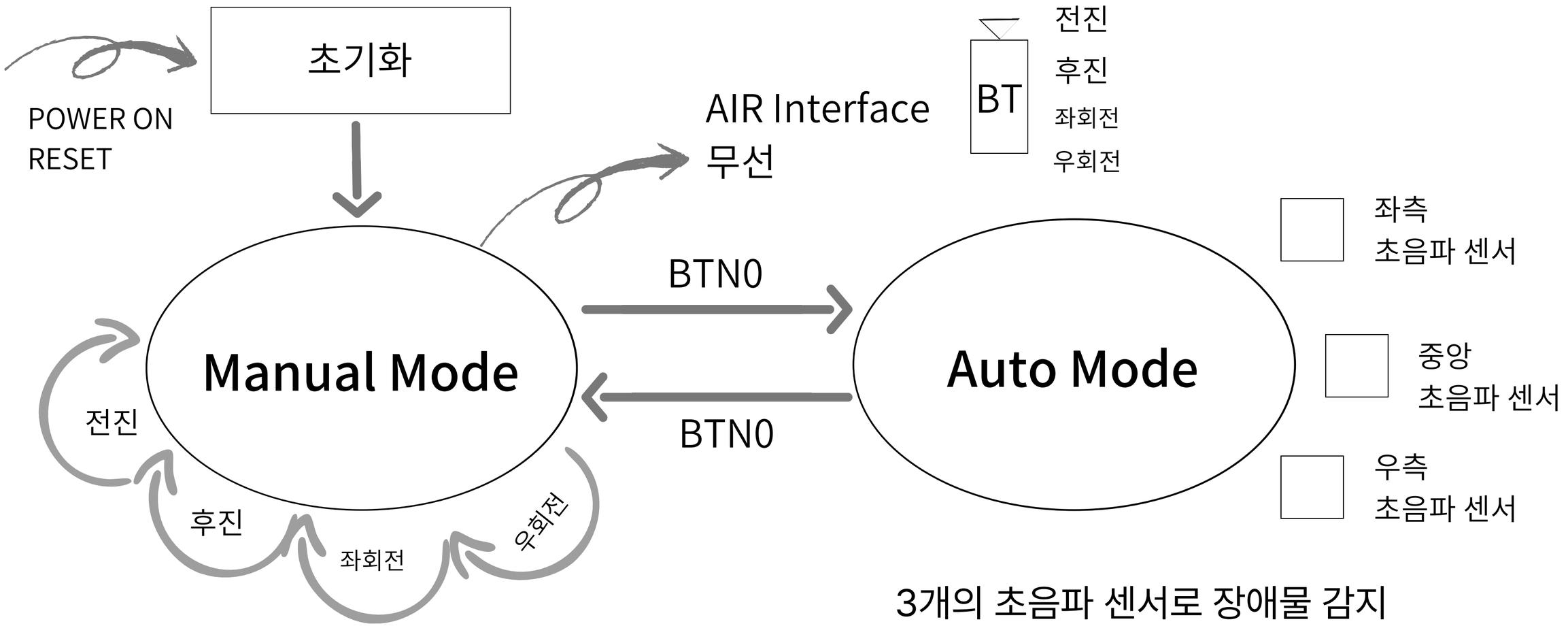


(출처: https://m.blog.naver.com/makeflood/222421800809)

Polling & Interrupt

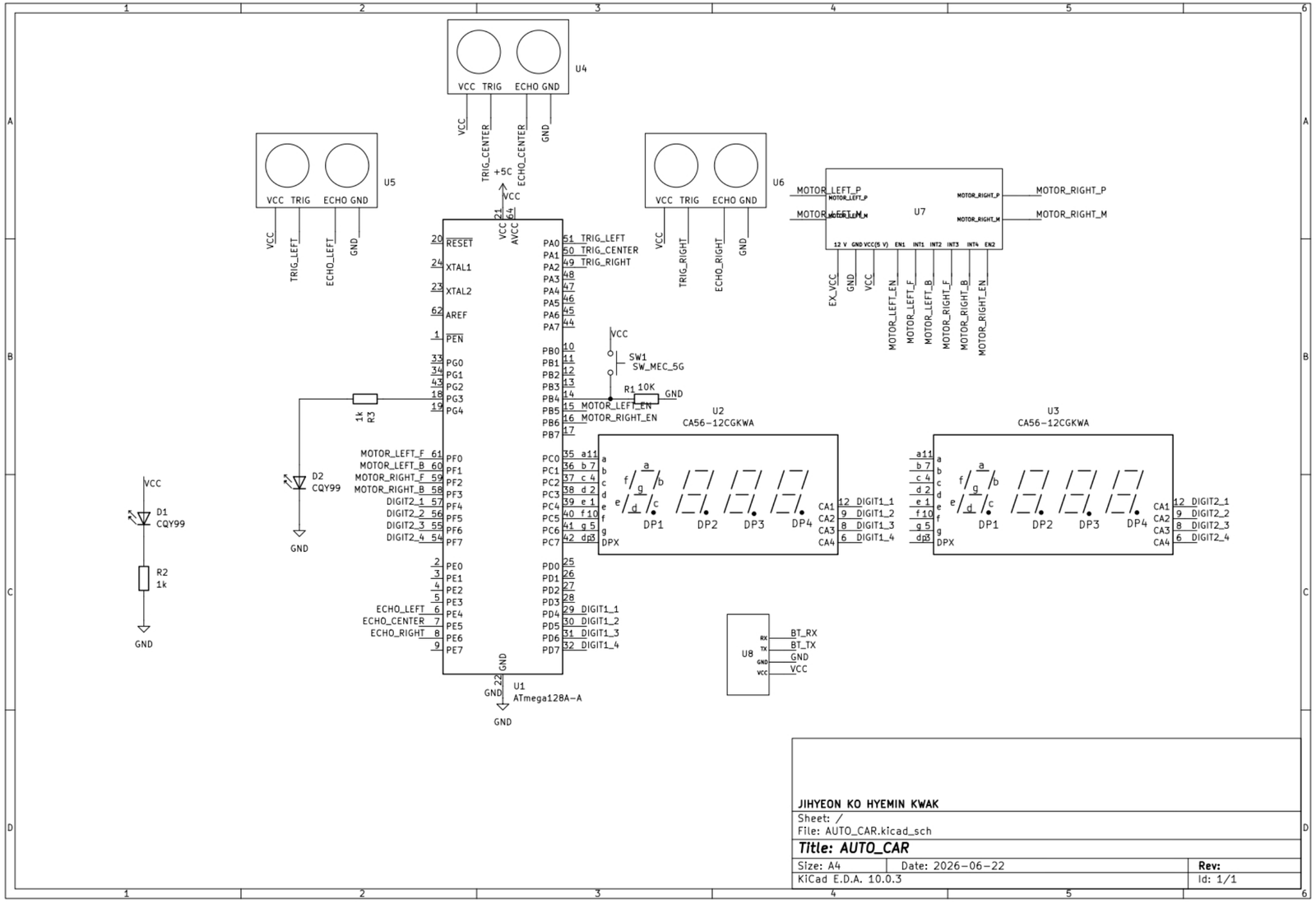
구분	Polling	Interrupt
이벤트 확인	CPU가 직접 확인	장치가 CPU 호출
CPU 사용률	높음	낮음
응답속도	느릴 수 있음	빠름
구현 난이도	쉬움	비교적 어려움
전력 소모	큼	적음
확장성	낮음	높음
실시간성	낮음	높음

STD(상태전이도)



3개의 초음파 센서로 장애물 감지

회로도


U4
VCC TRIG ECHO GND
VCC
TRIG_CENTER
+5C
ECHO_CENTER
GND
U5
VCC TRIG ECHO GND
VCC
TRIG_LEFT
ECHO_LEFT
GND
U6
VCC TRIG ECHO GND
VCC
TRIG_RIGHT
ECHO_RIGHT
GND
U7
MOTOR_LEFT_P
MOTOR_LEFT_M
MOTOR_RIGHT_P
MOTOR_RIGHT_M
12 V
GND
VCC(5 V)
EN1
INT1
INT2
INT3
INT4
EN2
EX_VCC
GND
VCC
MOTOR_LEFT_EN
MOTOR_LEFT_F
MOTOR_LEFT_B
MOTOR_RIGHT_F
MOTOR_RIGHT_B
MOTOR_RIGHT_EN
U1
ATmega128A-A
RESET
XTAL1
XTAL2
AREF
PEN
PG0
PG1
PG2
PG3
PG4
PF0
PF1
PF2
PF3
PF4
PF5
PF6
PF7
PE0
PE1
PE2
PE3
PE4
PE5
PE6
PE7
PA0
PA1
PA2
PA3
PA4
PA5
PA6
PA7
PB0
PB1
PB2
PB3
PB4
PB5
PB6
PB7
PC0
PC1
PC2
PC3
PC4
PC5
PC6
PC7
PD0
PD1
PD2
PD3
PD4
PD5
PD6
PD7
TRIG_LEFT
TRIG_CENTER
TRIG_RIGHT
MOTOR_LEFT_EN
MOTOR_RIGHT_EN
DIGIT1_1
DIGIT1_2
DIGIT1_3
DIGIT1_4
SW1
SW_MEC_5G
R1 10K
1k R3
D2 CQY99
D1 CQY99
R2 1k
U2
CA56-12CGKWA
U3
CA56-12CGKWA
DP1
DP2
DP3
DP4
DPX
CA1
CA2
CA3
CA4
DIGIT2_1
DIGIT2_2
DIGIT2_3
DIGIT2_4
U8
RX
TX
BT_RX
BT_TX
JIHYEON KO HYEMIN KWAK
Sheet: /
File: AUTO_CAR.kicad_sch
Title: AUTO_CAR
Size: A4
Date: 2026-06-22
Rev:
KiCad E.D.A. 10.0.3
Id: 1/1

main 함수

```
int main(void)
{
    init_uart1();
    init_button();
    init_motor_driver();
    init_timer1_pwm();
    init_ultrasonic();
    init_fnd();

    sei();   // 전역(대문) interrupt 허용

    while (1)
    {
        fnd_update();
        mode_button_check();
        if (func_state == MANUAL_MODE)
        {
            manual_mode();
        }
        else
        {
            auto_mode();
        }
        fnd_update();
    }
}
```

1

장치 초기화

DDR · Timer · UART · Interrupt 설정

2

버튼 확인

주행 중: 모드 전환 / 종료 후: 조회 전환

3

주행 함수 선택

AUTO면 센서 판단, MANUAL이면 명령 처리

4

FND 버퍼 갱신

현재 시간과 동작 상태를 8자리에 반영

manual_mode

```
void manual_mode(void)
{
    switch(bt_data) {
        case 'F':
        case 'f':
            forward(500);   // 4us x 500 = 0.002sec(2ms)
            break;
        case 'B':
        case 'b':
            backward(500);
            break;
        case 'L':
        case 'l':
            turn_left(700);  // 4us x 700 = 0.0028sec
            break;
        case 'R':
        case 'r':
            turn_right(700);
            break;
        case 'S':
        case 's':
            stop();
            fnd_finish_drive();
            func_state = MANUAL_MODE;
            bt_data = 0;
            break;
        default:
            break;
    }
}
```

명령

F **전진** forward(500)

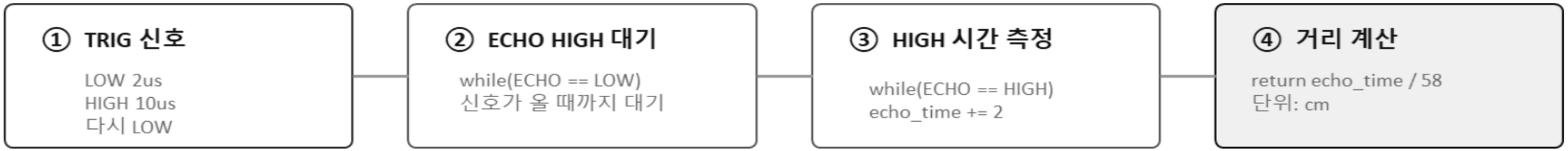
B **후진** backward(600)

L **좌회전** turn_left(700)

R **우회전** turn_right(700)

S **정지** stop()

초음파 센서




```
static int ultrasonic_read_cm(unsigned char trig_pin, unsigned char echo_
{
    unsigned long echo_time = 0;
    unsigned long timeout = 0;

    TRIG_PORT &= ~(1 << trig_pin); //초음파 트리거 초기화
    _delay_us(2);
    TRIG_PORT |= (1 << trig_pin); //트리거 high
    _delay_us(10);
    TRIG_PORT &= ~(1 << trig_pin); //트리거 Low

    while (!(ECHO_PINREG & (1 << echo_pin))) //에코 신호를 기다림
    {
        _delay_us(2);
        timeout += 2;
        if (timeout >= MAX_ECHO_US)
        {
            return 400; //지정한 시간 내에 돌아오지 않으면 400값을 리턴
        }
    }

    while (ECHO_PINREG & (1 << echo_pin)) //에코 신호가 high
    {
        _delay_us(2);
        echo_time += 2;
        if (echo_time >= MAX_ECHO_US)
        {
            return 400; // 지정한 시간보다 오래 걸렸으면 400리턴
        }
    }

    return (int)(echo_time / 58); //정상이면 시간 값을 리턴
}
```

왜 echo_time / 58 인가

초음파는 장애물까지 갔다가 다시 돌아오므로 측정 시간은 왕복
소리 속도를 cm/us 단위로 환산하고 왕복 시간을
반으로 나누면 약 58us당 1cm

정확한 측정을 위한

타임아웃: 약 30 ms 동안 ECHO가 없으면 400 반환

auto_mode

앞 거리 측정

front < 15

```
front_ultrasonic_distance = ultrasonic_read_cm(TRIG_FRONT, ECHO_FRONT);
_delay_ms(40);

if (front_ultrasonic_distance > FRONT_SAFE_CM)
{
    forward(AUTO_SPEED);
    return;

}
```

전진

안전하면 바로 이동

정지 후 좌우 측정

장애물 감지

```
    stop();
    _delay_ms(80);

    left_ultrasonic_distance = ultrasonic_read_cm(TRIG_LEFT, ECHO_LEFT);
    _delay_ms(40);
    right_ultrasonic_distance = ultrasonic_read_cm(TRIG_RIGHT, ECHO_RIGHT);
    _delay_ms(40);
```

측면 충돌 판단

10cm미만이면 반대 회전

```
if (left_ultrasonic_distance < SIDE_CRASH_CM)
{
    turn_right(TURN_SPEED); // 측면 회피는 딜레이를 짧게
    _delay_ms(350);
    return;
}
else if (right_ultrasonic_distance < SIDE_CRASH_CM)
{
    turn_left(TURN_SPEED);
    _delay_ms(350);
    return;
}
```

한 쪽 막힘

장애물을 감지 한 반대 쪽으로 회전

```
// 앞 + 왼쪽 막힘 (<= 로 수정하여 정확히 10일 때의 버그 방지)
if (left_ultrasonic_distance <= SIDE_SAFE_CM && right_ultrasonic_distance > SIDE_SAFE_CM)
{
    turn_right(TURN_SPEED);
    _delay_ms(350);
    return;
}

// 앞 + 오른쪽 막힘
if (right_ultrasonic_distance <= SIDE_SAFE_CM && left_ultrasonic_distance > SIDE_SAFE_CM)
{
    turn_left(TURN_SPEED);
    _delay_ms(400);
    return;
}
```

양쪽 막힘

후진 후 넓은쪽으로 회전

```
// 앞 + 좌우 모두 막힘: 후진 후 넓은 쪽으로 회전
if (left_ultrasonic_distance <= SIDE_SAFE_CM && right_ultrasonic_distance <= SIDE_SAFE_CM)
{
    backward(AUTO_SPEED);
    _delay_ms(350);

    stop();
    _delay_ms(80);

    if (left_ultrasonic_distance > right_ultrasonic_distance)
    {
        turn_left(TURN_SPEED);
        _delay_ms(400);
        return;
    }
    else
    {
        turn_right(TURN_SPEED);
        _delay_ms(400);
        return;
    }
}
```

양쪽 넓음

더 넓은쪽으로 회전

```
    // 위 모든 조건에 해당하지 않으면 (좌우 모두 꽤 넓음), 더 넓은 쪽으로 회전
    if (left_ultrasonic_distance > right_ultrasonic_distance)
    {
        turn_left(TURN_SPEED);
        _delay_ms(250);
        return;
    }
    else
    {
        turn_right(TURN_SPEED);
        _delay_ms(250);
        return;
    }
```

주행 중 fnd display

위 FND 4자리

주행 시간이 초 단위로 표시

예: 0007 → 7초

아래 FND 4자리

첫 자리: F, b, L, r, S
나머지: 원형 애니메이션

Timer2 ISR

index 0~7 반복
fnd_ms_count 증가
정지 중에는 시간 증가 X

```
static void fnd_display_running(void)
{
    unsigned char circle_step = (fnd_ms_count / 120) % 6;

    // 위 4자리: 주행시간
    fnd_set_4digit(top_data, fnd_run_sec);

    // 아래 4자리: 방향 문자 + 바로 옆 서클 애니메이션
    bottom_data[0] = motion_seg(current_motion);
    bottom_data[1] = circle_seg(circle_step);
    bottom_data[2] = circle_seg(circle_step);
    bottom_data[3] = circle_seg(circle_step);
}
```

```
void fnd_set_motion(motion_t motion)
{
    current_motion = motion;

    if (motion != MOTION_STOP && motion != last_count_motion)
    {
        if (motion == MOTION_BACKWARD)
        {
            fnd_backward_count++;
        }
        else if (motion == MOTION_LEFT)
        {
            fnd_left_count++;
        }
        else if (motion == MOTION_RIGHT)
        {
            fnd_right_count++;
        }

        last_count_motion = motion;
    }

    if (motion == MOTION_STOP)
    {
        last_count_motion = MOTION_STOP;
    }
}
```

주행 결과 fnd display

```
static void fnd_display_result(void)
{
    unsigned int value = 0;
    unsigned char label = 0;
    unsigned char sreg;

    if (result_page == 0)
    {
        label = SEG_D | SEG_E | SEG_F | SEG_G;  // t, 주행시간
        value = fnd_run_sec;
    }
    else if (result_page == 1)
    {
        label = SEG_C | SEG_D | SEG_E | SEG_F | SEG_G;  // b, 후진 횟수
        value = fnd_backward_count;
    }
    else if (result_page == 2)
    {
        label = SEG_D | SEG_E | SEG_F;  // L, 좌회전 횟수
        value = fnd_left_count;
    }
    else
    {
        label = SEG_E | SEG_G;  // r, 우회전 횟수
        value = fnd_right_count;
    }

    // 종료 후: 위 4자리는 값, 아래 첫 자리는 항목 문자
    fnd_set_4digit(top_data, value);
    fnd_set_bottom_label(label);
}
```

페이지	아래 FND 라벨	위 FND 값	의미
page 0	t	fnd_run_sec	주행 시간
page 1	b	fnd_backward_count	후진 횟수
page 2	L	fnd_left_count	좌회전 횟수
page 3	r	fnd_right_count	우회전횟수

4번째 페이지 이후 버튼을 한 번 더 누르면 기록이 초기화되고 MANUAL_MODE로 복귀

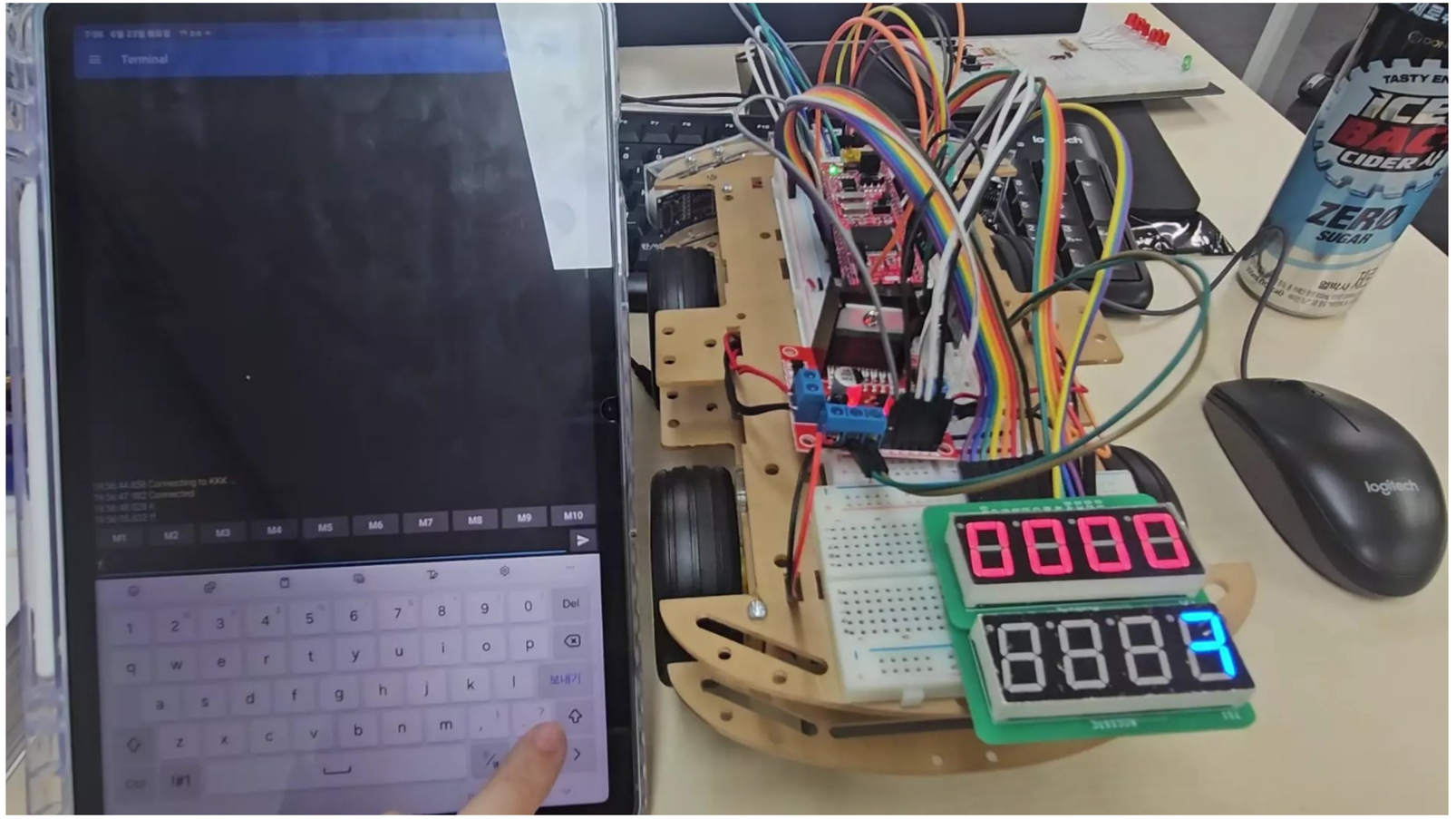
동작 영상

Auto_Mode (초음파센서 활용 자율 주행)



동작 영상

Manual_Mode (Bluetooth)



정리

입력

버튼: 모드 전환/결과 조회
UART1: 블루투스 문자 수신

판단

초음파 3개로 거리 측정
앞 장애물 감지 후 좌우 비교
측면 충돌 시 반대 방향 회피

출력

PWM으로 좌우 바퀴 속도 제어
FND로 시간·상태·결과 표시

센서 입력을 바탕으로 주행 상황을 판단하고,
모터 제어와 FND 표시로 동작 결과를 출력해 자율 주행 자동차를 완성했다.

시사점

본 프로젝트에서는 ATmega128A 기반 자율주행 RC카를 구현하였다.

수동 주행과 자율주행 모드 전환 기능을 통해 다양한 주행 상황에 대응할 수 있도록 하였으며, 초음파 센서를 활용한 실시간 장애물 감지 및 회피 기능을 적용하였다.

구현 과정에서 센서 입력, 판단 로직, 모터 및 FND 출력 제어가 함께 동작해야 자율주행 기능이 완성된다는 점을 배웠다.

이를 통해 임베디드 시스템에서 입력-판단-출력 구조의 중요성을 확인하였다.